$$\mathcal{D}[\![\tau]\!] = \begin{cases} \mathbf{fin}(|B|+1) & \text{if } \tau = \mathbf{float}[B;V] \text{ and } B \neq \top \\ \mathbf{float}[B;V] & \text{if } \tau = \mathbf{float}[B;V] \text{ and } B = \top \\ \text{(straightforward recursion on type structure)} & \text{otherwise} \end{cases}$$

Fig. 14. Transformation of types under $\mathcal{D}$, where $\mathcal{D}$ acts on types.

discretization process to discretizing expressions with *constant parameters only*, the remaining steps are completely analogous to Theorem 5.1.

Crucially, the *recoverability* constraints in the Continuous-rules from Figure 6 ensure that the parameter expressions of the respective sampling expressions can take only finitely many values, and that the cut sets are fine-grained enough to recover the values the parameter expressions evaluate to, as demonstrated in the preceding example.

A similar intuition applies to discretizable comparisons $e_1 < e_2$, where *both* expressions are non-constant but finite-cut. The *answers* constraints in the rule Less (resp. Less-Equal) ensure that at least one of e_1 and e_2 evaluates to only finitely many values, which enables discretization:

Example 5.3. Consider the following typed expression $(e_1 \leq e_2) : \mathbf{bool}$, given by

```
(if discrete(0.5, 0.5) <#2 1#2 then
       uniform(0,10) : float[B; T] else 2.5 : float[B; {2.5}])
<=
(if discrete(0.5, 0.5) <#2 1#2
       then 2.5 : float[B; {2.5}] else 3.5 : float[B; {3.5}])
```

where $B = \{\leq 2.5, \leq 3.5\}$, and where e_1 : `float[B; T]` and e_2 : `float[B; {2.5,3.5}]`. Intuitively, e_1 samples from a mixture of a uniform and a dirac distribution, and e_2 samples from a finite distribution. Why can $e_1 \leq e_2$ be discretized even though neither e_1 nor e_2 is a constant? As suggested by its type, e_2 can take only finitely many values, namely $\{2.5, 3.5\}$. We apply $\mathcal{D}[\![\cdot]\!]$:

```
(if discrete(0.5, 0.5) <#2 1#2 then discrete(0.25, 0.1, 0.65) else 0#3)
<=
(if discrete(0.5, 0.5) <#2 1#2 then 0#3 else 1#3)
```

Notice that the probability of e_1 evaluating to some value in $(-\infty, 2.5]$ indeed *coincides* with the probability of the discretization of e_1 evaluating to $0_{\#3}$. For e_2, on the other hand, the probability of it evaluating to 2.5 (resp. 3.5) *coincides* with the probability of its discretization evaluating to $0_{\#3}$ (resp. $1_{\#3}$). For this reason, we obtain a semantically equivalent discretization of $e_1 \leq e_2$.

5.0.1 Discretization of Types. We conclude by extending $\mathcal{D}[\![\cdot]\!]$ to types. Figure 14 shows how type τ is discretized to type $\mathcal{D}[\![\tau]\!]$. We then obtain the following natural result, proved in §E.1:

Theorem 5.4 (Discretization Is Well-Typed). *Let $\Gamma \vdash e : \tau$. We have $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]$, where $\mathcal{D}[\![\Gamma]\!]$ transforms all types in Γ, i.e., $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\Gamma(x)]\!]$ for all $x \in \mathrm{dom}(\Gamma)$.*

6 Operational Semantics and Soundness

We now establish the soundness of our approach in the following sense: For *every* expression e : **bool**, possibly including *continuous sampling, higher-order functions, recursion, and conditioning*, formulate exactness for the *uniform fragment*: the syntax in Figure 5 without sequencing, non-strict comparisons, finite comparisons in source programs, or continuous samplers other than **uniform**. Write $Expr_\tau^{\cup}$ for its well-typed expressions and $Expr_\tau^{\cup+}$ for its operational closure under the finite comparisons and dispatch syntax emitted by Figure 13. The conditional claim is restricted to closed e : **bool** in $Expr_{\mathbf{bool}}^{\cup}$:

*e : **bool** If the emitted core term satisfies the measurable weak compiler-simulation, administrative-normalization, and $\mathcal{D}[\![e : \mathbf{bool}]\!]$: **bool** (obtained from Figure 13) terminal-limit compatibility obligations stated in the appendix, then e : **bool** and $\mathcal{D}[\![e : \mathbf{bool}]\!]$: **bool** are semantically equivalent.*

The first step is to make formal sense of semantic equivalence . For that, we will define through a monadic operational semanticsfor our language in §6.1. We will then proceed by proving the above claim. This is non-trivial: discretization decisions are driven by non-local, type-propagated information, so we cannot proceed by a local rule-by-rule argument. To prove the claim rigorously, we relate the executions of e and $\mathcal{D}[\![e]\!]$ through a coupling-style logical relation in §6.1. . The remaining proof obligation is nontrivial: the emitted target takes deterministic administrative steps that have no matching source step, and substitution can change distribution dispatch. A proof therefore needs the weak compiler simulation described in §F; the earlier synchronous graph is insufficient.

6.1 Operational Semantics

The type-directedness of our discretization function $\mathcal{D}[\![\cdot]\!]$ requires our soundness argument to depend on the types of the given (sub-)expressions as well. We thus start by defining a *type-dependent semantic functional*, i.e., we define for every type τ a function $\langle\cdot\rangle\colon Expr_\tau \to \mathcal{D}(Vals_\tau + \bot)$, mapping each *fully type-annotated* expression $e : \tau$ to a *sub*-probability measure $\langle e : \tau\rangle$ over values of type τ or $\bot$. In the presence of possibly unbounded recursion *and* conditioning, our semantics has to distinguish between non-termination and observation failure, which we realize as follows: The "missing" probability mass in $\langle e : \tau\rangle$ is the probability that e diverges. The probability of $\bot$ is the probability of encountering an observation failure. With this model, semantic equivalence indeed captures our sought-after correctness claim: If $\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle$, then both expressions have the same probability of producing true, false, or an observation failure.

6.1.1 Small Step Semantics. The type-dependent semantic functional $\langle\cdot\rangle$ is defined via a *small-step* semantic functional $[\![\cdot]\!]\colon Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$. Defining the latter requires us to carefully design a suitable measurable space over the set of type-annotated expressions of type τ and $\bot$. All details on these measure-theoretic constructions are presented in §F.2. Let us consider the key idea here.

Let $\tau = \mathbf{bool}$ for the sake of concreteness. Care must be taken since, e.g., sub-expressions of type $\mathbf{float}[B; \{3.5, 4.5\}]$ cannot evaluate to *arbitrary* reals but are restricted to the values $\{3.5, 4.5\}$. We tackle this by extracting from each type-annotated expression a suitable *skeleton*, where float constants are replaced by "holes" (hence the term "skeleton"). For instance, the expression

$$e \quad = \quad (x : \mathbf{float}[\{<3.5, <4.5\}; \top] \;<\; y : \mathbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \mathbf{bool}$$

gives rise to the skeleton

$$s \quad = \quad (\square : \mathbf{float}[\{<3.5, <4.5\}; \top] \;<\; \square : \mathbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \mathbf{bool}$$

and the type annotations tell us that the possible values these holes can be filled with are in $\mathbb{R} \times \{3.5, 4.5\}$. We can thus inherit a measurable space over all well-typed expressions arising from this particular skeleton s from the standard Borel measurable space over $\mathbb{R} \times \{3.5, 4.5\}$. The measurable space over *all* well-typed expressions of type $\mathbf{bool}$ (i.e., over $Expr_{\mathbf{bool}}$) is then obtained from the measurable spaces of *all* skeletons that give rise to expressions of type $\mathbf{bool}$.

Equipped with this family of measurable spaces, we define the small-step functional $[\![\cdot]\!]$ in the expected, type-preserving manner (cf. Tables 1 and 2). For instance, if e_1 is not a value, then[1]

$$[\![(\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2) : \tau_2]\!] \;=\; \underbrace{[\![e_1 : \tau_1]\!] \gg\!= \lambda g \in Expr_{\tau_1}.\delta_{(\mathbf{let}\ x=g\ \mathbf{in}\ e_2:\tau_2):\tau_2}}_{\text{standard probabilistic monadic bind over suitable sub-Markov kernels}} ,$$

i.e., $[\![e_1 : \tau_1]\!]$ yields the sub-distribution over expressions obtained from executing e_1 for *one step*, and the monadic bind $\gg\!=$ "folds" this distribution into the surrounding $\mathbf{let}$-statement. The intuition for the remaining statements is analogous. Crucially, $\gg\!=$ propagates the probability of encountering $\bot$ — an observation failure — through executions. For instance, assume that e_1 from above is given by $\mathbf{observe}\ (\text{false} : \mathbf{bool}) : \mathbf{unit}$. Since e_1 encounters an observation failure with probability 1, i.e., $[\![\mathbf{observe}\ (\text{false} : \mathbf{bool}) : \mathbf{unit}]\!](\{\bot\}) = 1$, the same holds for the surrounding $\mathbf{let}$-statement, i.e., also $[\![(\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2) : \tau_2]\!](\{\bot\}) = 1$. All details — including proofs that all monadic binds are well-behaved since they operate on suitable Markov kernels

[1] Put formally, $(\mu \gg\!= f)(A) = \int f(x)(A) \cdot \mu(dx)$for a sub-Markov kernel $f : X \to \mathcal{D}(Y + \bot)$, $(\mu \gg\!= f)(A) = \int_{X+\bot} f_\bot(x)(A)\, \mu(dx)$, where $f_\bot(\bot) = \delta_\bot$; see Definition F.29for details. δ_e denotes the Dirac distribution of e.

— are provided The appendix proves the generic strict-bind results and records the remaining operational measurability obligations in §F.4.

6.1.2 From Small-Step to Big-Step Semantics. We now define $\langle\cdot\rangle$ via $[\![\cdot]\!]$. For that, we Put $S_\tau(e') = [\![e' : \tau]\!]$ for ordinary expressions $e' \in Expr_\tau$. We first lift the one-step semantics $[\![\cdot]\!]$ to an n-step semantics $\langle\cdot\rangle_n$ in the expected wayusing the canonical extension $(S_\tau)_\perp$:

$$\langle e : \tau\rangle_0 = \delta_{e:\tau} \qquad \text{and} \qquad \langle e : \tau\rangle_{n+1}(A) \;=\; \int_{Expr_\tau+\perp} (S_\tau)_\perp(e')(A)\, \langle e : \tau\rangle_n(de').$$

This construction is well-defined since the functional Conditional on the operational measurability obligations stated in the appendix, this construction is well defined because $[\![\cdot]\!]$ itself gives rise to a suitable Markov is a sub-Markov kernel (Lemma F.40). We then define for every type τ the functional $\langle\cdot\rangle\colon Expr_\tau \to \mathcal{D}(Vals_\tau + \perp)$ as $\langle e : \tau\rangle = \lambda A.\, \lim_{n\to\infty}\langle e : \tau\rangle_n(A)$, which is well-defined by the monotone convergence theorem: the sequence $(\langle e : \tau\rangle_n)_{n\in\mathbb{N}}$ is monotonically increasing The full finite-step measures on expressions are not increasing: a step moves mass away from its source expression. Following the terminal-limit construction in the appendix, let ν^e_n be the restriction of $\langle e : \tau\rangle_n$ to the measurable terminal subspace $Vals_\tau + \perp$. Terminal outcomes are absorbing, so $\nu^e_n(A) \le \nu^e_{n+1}(A)$ for every measurable set terminal event Aover $Vals_\tau + \perp$ (cf. Lemma F.42) and upper-bounded by 1: executing a program for more and more steps can only increase the probability of terminating in a given set of values or an observation failure. We define

$$\langle e : \tau\rangle(A) = \lim_{n\to\infty} \nu^e_n(A).$$

The appendix proves eventwise monotonicity and countable additivity of this limit.

6.2 Soundness via Probabilistic Couplings

We first state the central theorem, then outline our coupling-style correctness argument. state the result supplied by a completed weak simulation. The appendix records the exact missing obligation.

Proposition 6.1 (Conditional exactness). *Let e : **bool**. Then e : **bool** $\in Expr^{\cup}_{\mathbf{bool}}$ be closed. Assume the operational measurability obligations, a measurable weak target kernel, and a measurable weak coupling whose marginals are the source step and target weak step. Assume generated administrative reduction is normalizing, the restrictions of the coupling iterates to terminal related outcomes have an increasing limit, and the target projection of that limit is the ordinary target terminal semantics. If related terminal Boolean values agree and the coupling preserves observation-failure mass, then* $\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle$.

Intuitively, we prove that the two programs can execute in a synchronized manner such that their probabilities of terminating in value v coincide for each $v \in \{\text{true}, \text{false}, \perp\}$. Under these terminal-limit compatibility premises, the limiting coupling has the two ordinary terminal semantics as its component pushforwards. Agreement of related terminal Booleans and preservation of observation-failure mass make the pushforwards agree on true, false, and $\perp$. The required weak simulation, administrative-normalization proof, and terminal-limit compatibility theorem have not yet been established for the literal emitted core term.

7 Implementation, Case Studies, and Evaluation

We have implemented the Slice type inference and discretization procedures in OCaml. The implementation is open-source and available at https://github.com/[anonymized], together with all benchmarks, scripts, and a Docker image for reproduction. To compute the interval masses of continuous distributions, we use the OCaml GSL package [19] for cumulative distribution function calculations. All experiments were run in this Docker image on an Apple M3 with 16 GB of RAM.

This section is driven by two research questions:

(1) Does Slice enable exact inference on programs combining continuous sampling, higher-order functions, and unbounded recursion?
(2) On programs that are within the scope of existing tools for exact inference, is the combination of Slice and discrete inference back-ends competitive?

Proof. We first verify that i) (X, Σ_X) is a measurable space, then show that ii) f induces an isomorphism of measurable spaces $(X, \Sigma_X) \cong (Y, \Sigma_Y)$.

Proof of i) We show that $\Sigma_X = f^{-1}(\Sigma_Y)$ is a σ-algebra on X.

- **(Contains X):** $X = f^{-1}(Y) \in \Sigma_X$ since $Y \in \Sigma_Y$.
- **(Complements):** If $f^{-1}(A) \in \Sigma_X$, then $X \setminus f^{-1}(A) = f^{-1}(Y \setminus A) \in \Sigma_X$ since $Y \setminus A \in \Sigma_Y$.
- **(Countable unions):** If $f^{-1}(A_i) \in \Sigma_X$ for $i \in \mathbb{N}$, then $\bigcup_i f^{-1}(A_i) = f^{-1}(\bigcup_i A_i) \in \Sigma_X$ since $\bigcup_i A_i \in \Sigma_Y$.

Hence (X, Σ_X) is a measurable space.

Proof of ii) Two measurable spaces (X, Σ_X) and (Y, Σ_Y) are isomorphic iff there exists a bijection $f : X \to Y$ such that f and f^{-1} are measurable.

We first show that f is measurable. Measurability of f means that $f^{-1}(A) \in \Sigma_X$ for every $A \in \Sigma_Y$. Let $A \in \Sigma_Y$ be arbitrary. By definition of Σ_X,

$$f^{-1}(A) \in f^{-1}(\Sigma_Y) = \Sigma_X.$$

It follows that f is measurable.

Next we show that f^{-1} is measurable. Measurability of f^{-1} means that $(f^{-1})^{-1}(B) \in \Sigma_Y$ for every $B \in \Sigma_X$. Let $B \in \Sigma_X$ be arbitrary. By definition of Σ_X, there exists some $A \in \Sigma_Y$ such that $B = f^{-1}(A)$.

Observe that:

$$\begin{aligned} (f^{-1})^{-1}(B) &= f(B) && \text{(preimage of } f^{-1} \text{ is } f) \\ &= f(f^{-1}(A)) && \text{(substituting } B = f^{-1}(A)) \\ &= A && (f \text{ is a bijection}) \end{aligned}$$

Since $A \in \Sigma_Y$, it follows that f^{-1} is measurable.

Therefore f is a measurable bijection with measurable inverse, so it is an isomorphism of measurable spaces. □

Lemma F.4 (Minimality of generated σ-algebras). *Let X be a set and let $\mathcal{G} \subseteq \mathcal{P}(X)$. Suppose $\mathcal{C}$ is a σ-algebra on X such that $\mathcal{G} \subseteq \mathcal{C}$. Then*

$$\sigma(\mathcal{G}) \subseteq \mathcal{C}.$$

Proof. Recall that $\sigma(\mathcal{G})$ is defined as the smallest σ-algebra on X containing $\mathcal{G}$, given concretely by the intersection of all σ-algebras on X containing $\mathcal{G}$:

$$\sigma(\mathcal{G}) = \bigcap_{\substack{\mathcal{F} \text{ is a } \sigma\text{-algebra on } X \\ \mathcal{G} \subseteq \mathcal{F}}} \mathcal{F}.$$

Since $\mathcal{C}$ is a σ-algebra on X containing $\mathcal{G}$, it appears in this intersection. Therefore,

$$\sigma(\mathcal{G}) \subseteq \mathcal{C}.$$

□

Definition F.5 (Sub-Markov kernels). Given measurable spaces (X, Σ_X) and (Y, Σ_Y), a Markov sub-Markov kernel with source X and target Y is a function $P : X \times \Sigma_Y \to [0, 1]$ (or equivalently, $X \to \Sigma_Y \to [0, 1]$) that maps each source state $x \in X$ to a subprobability distribution over target states $P(x, -) : \Sigma_Y \to [0, 1]$.

We call P a *Markov sub-Markov kernel* if it satisfies two conditions:

(1) *Probability Subprobability measure:* For each source state $x \in X$, the map $A \mapsto P(x, A)$ must be a probability measuresubprobability measure; in particular, $P(x, Y) \leq 1$.
(2) *Measurable function:* For each event $A \in \Sigma_Y$ in the target space, the map $x \mapsto P(x, A)$ must be a measurable function.

If additionally $P(x, Y) = 1$ for every $x \in X$, then P is a *Markov kernel*. Thus every Markov kernel is a sub-Markov kernel, but the converse need not hold.

(iii) Closure under countable unions: Let $(C_n)_{n\in\mathbb{N}}$ be a sequence with $C_n \in \mathcal{F}_{Expr_\tau+\bot}$ for all n. For each n, define $A_n := C_n \cap Expr_\tau \in \mathcal{F}_{Expr_\tau}$, where the latter holds since C_n is of the form A or $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$, and in either case $C_n \cap Expr_\tau = A$.

Let $A := \bigcup_{n\in\mathbb{N}} A_n$. Since $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $A \in \mathcal{F}_{Expr_\tau}$. Now observe:

$$\bigcup_{n\in\mathbb{N}} C_n = A \cup \left(\bigcup_{n\in\mathbb{N}} (C_n \setminus Expr_\tau)\right).$$

The second factor is either $\emptyset$ (if every $C_n = A_n$) or $\{\bot\}$ (if some $C_n = A_n \cup \{\bot\}$). In the first sub-case, $\bigcup_n C_n = A$ is of the form A for some $A \in \mathcal{F}_{Expr_\tau}$; in the second, $\bigcup_n C_n = A \cup \{\bot\}$ is of the form $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$. In both cases $\bigcup_n C_n \in \mathcal{F}_{Expr_\tau+\bot}$. □

Finally, we can equip this measurable space $(Expr_\tau + \bot, \mathcal{F}_{Expr_\tau+\bot})$ with a subprobability measure such that $\mu(Expr_\tau + \bot) \le 1$.

We give some basic lemmas:

Lemma F.27. *Let $(Expr_\tau + \bot, \mathcal{F}_{Expr_\tau+\bot})$ be the Borel measurable space of well-typed expressions with error from Definition F.25. Let $Vals_\tau \subset Expr_\tau$ be the set of all well-typed value expressions. Let $N_\tau \subset Expr_\tau$, defined as $N = Expr_\tau \setminus Vals_\tau$, be the set of all well-typed non-value expressions. Let $\{\bot\}$ be the singleton set containing the error element $\bot$. The following are true:*

(1) The set $Vals_\tau \in \mathcal{F}_{Expr_\tau+\bot}$.
(2) The set $\{\bot\} \in \mathcal{F}_{Expr_\tau+\bot}$.
(3) The set $N_\tau \in \mathcal{F}_{Expr_\tau+\bot}$.

Proof. We prove the measurability of $Vals_\tau$, $\{\bot\}$, and N_τ in separate parts. For each, we show that it belongs to the σ-algebra $\mathcal{F}_{Expr_\tau+\bot}$ on the extended expression space $Expr_\tau + \bot$.

(1) **(Measurability of $Vals_\tau$):** Every expression skeleton $s \in Skel_\tau$ is either a value skeleton or a non-value skeleton, depending on whether the syntactic form of s is a value. An expression $e \in Expr_\tau$ is a value iff its skeleton is a value skeleton.
To show $Vals_\tau \in \mathcal{F}_{Expr_\tau}$, by Definition F.23, it suffices to show that $\forall s \in Skel_\tau, Vals_\tau \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$. Fix $s \in Skel_\tau$:
- If s is a value skeleton, then every expression in $Expr_{s;\tau}$ is a value, so $Vals_\tau \cap Expr_{s;\tau} = Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$.
- If s is a non-value skeleton, then no expression in $Expr_{s;\tau}$ is a value, so $Vals_\tau \cap Expr_{s;\tau} = \emptyset \in \mathcal{F}_{Expr_{s;\tau}}$.

Thus, $Vals_\tau \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$ for any $s \in Skel_\tau$, so $Vals_\tau \in \mathcal{F}_{Expr_\tau} \subseteq \mathcal{F}_{Expr_\tau+\bot}$.

(2) **(Measurability of $\{\bot\}$):** Taking $A = \emptyset$ in Definition F.25 gives $\{\bot\} \in \mathcal{F}_{Expr_\tau+\bot}$.

(3) **(Measurability of N_τ):** $N_\tau = Expr_\tau \setminus Vals_\tau$ is the complement of $Vals_\tau$ in $Expr_\tau$. Since $Vals_\tau \in \mathcal{F}_{Expr_\tau}$ and $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $N_\tau \in \mathcal{F}_{Expr_\tau} \subseteq \mathcal{F}_{Expr_\tau+\bot}$.

□

Note that in general, given (X, Σ_X) and (Y, Σ_Y) to be measurable spaces, $((X \times Y) + \bot)$ forms a measurable space, since the product $(X \times Y)$ is measurable and for any measurable space (Z, Σ_Z), the space $(Z + \bot, \Sigma_{Z+\bot})$ with $\Sigma_{Z+\bot}$ generated by $\Sigma_Z \cup \bot$ is measurable.

F.3 Distribution Monad Strict Sub-Markov Kleisli Composition with Error

Let X be a set equipped with a suitable measurable space so that $\mathcal{D}(X + \bot)$ denotes the set ofprobability measures over $X + \bot$. We will make heavy use of the following generic monad construction, for which one easily verifies the monad laws: measurable space and equip $X + \bot$ with the sigma algebra generated by the measurable subsets of X and the singleton $\{\bot\}$. Write $\mathcal{D}(X + \bot)$ for the subprobability measures on this measurable space.

We define where δ_x denotes the Dirac measure for x, and f is a Markov kernel.

Definition F.28 (⊥-extension). Define the ⊥-extension of $g : X \to \mathcal{D}(Y + \bot)$ to be $g_\bot \colon (X + \bot) \to \mathcal{D}(Y + \bot)$, obtained from g by:

$$g_\bot(e) = \begin{cases} \delta_\bot & \text{if } e = \bot \\ g(e) & \text{otherwise .} \end{cases}$$

Thus the added error point is absorbing. A continuation is specified only on ordinary inputs; its value at ⊥ is not a free choice.

Definition F.29 (Strict sub-Markov Kleisli bind). For measurable spaces X, Y, define

$$\mathsf{return}_X : X \to \mathcal{D}(X + \bot), \qquad \mathsf{return}_X(x) = \delta_x,$$
$$\mathsf{bind}_{X,Y} : \mathcal{D}(X + \bot) \to \big(X \to \mathcal{D}(Y + \bot)\big) \to \mathcal{D}(Y + \bot),$$
$$\mathsf{bind}_{X,Y}(\mu)(g)(A) = \int_{X+\bot} g_\bot(x)(A)\, \mu(dx),$$

where g is required to be a sub-Markov kernel and A is measurable in $Y + \bot$. We write $\mu \gg\!= g$ only for this typed operation. By definition, it always integrates the canonical ⊥-extension $g_\bot$, so

$$\delta_\bot \gg\!= g = \delta_\bot.$$

In particular, bind cannot recover from an error. By Lemma F.30, the integrand is a sub-Markov kernel, and the result is a subprobability measure because

$$(\mu \gg\!= g)(Y + \bot) = \int_{X+\bot} g_\bot(x)(Y + \bot)\, \mu(dx) \le \mu(X + \bot) \le 1.$$

Lemma F.30 (⊥-extension is a sub-Markov kernel). *Given a Markov sub-Markov kernel $f : X \to \mathcal{D}(Y + \bot)$, its ⊥ extension $f_\bot : (X + \bot) \to \mathcal{D}(Y + \bot)$ is also a Markov kernelsub-Markov kernel. If f is a Markov kernel, then $f_\bot$ is a Markov kernel.*

Proof. We verify the two conditions for $f_\bot$ to be a Markov sub-Markov kernel.

(1) Probability Subprobability measure: We show for each $e \in X + \bot$, the map

$$A \mapsto f_\bot(e)(A)$$

is a probability subprobability measure.

Fix $e \in X + \bot$.

- If $e = \bot$, then $f_\bot(\bot) = \delta_\bot$, the Dirac measure at ⊥, which is a probability measure on $Y + \bot$.
- If $e \in X$, then $f_\bot(e) = f(e)$, and since f is a Markov sub-Markov kernel by assumption, $f(e)$ is a probability subprobability measure on $Y + \bot$.

Thus, for all $e \in X + \bot$, the map $A \mapsto f_\bot(e)(A)$ is a probability subprobability measure.

(2) Measurable function: We show for each $A \in \mathcal{F}_{Y+\bot}$, the map $e \mapsto f_\bot(e)(A)$ is a measurable function.

Fix $A \in \mathcal{F}_{Y+\bot}$. Define

$$g \colon (X + \bot) \to [0, 1], \qquad g(e) := f_\bot(e)(A).$$

We prove g is measurable by showing $g^{-1}(B) \in \mathcal{F}_{X+\bot}$ for every $B \in \mathcal{B}([0, 1])$. It suffices to prove that for every $a \in \mathbb{R}$,

$$g^{-1}((a, \infty) \cap [0, 1]) \in \mathcal{F}_{X+\bot}$$

because the family of sets $\{(a, \infty) \cap [0, 1] : a \in \mathbb{R}\}$ generates $\mathcal{B}([0, 1])$.

Because $g(e) \in [0, 1]$ for all $e \in X + \bot$ by definition, we have for every $a \in \mathbb{R}$,

$$\begin{aligned} g^{-1}\big((a, \infty) \cap [0, 1]\big) &= \{e \in X + \bot \mid g(e) \in (a, \infty) \cap [0, 1]\} \\ &= \{e \in X + \bot \mid (g(e) \in (a, \infty)) \wedge (g(e) \in [0, 1])\} \\ &= \{e \in X + \bot \mid (g(e) > a) \wedge (g(e) \in [0, 1])\} \\ &= \{e \in X + \bot \mid g(e) > a\}, \end{aligned}$$

where the last step uses that $g(e) \in [0, 1]$ always holds. Hence it suffices to show that $\{e \in X + \bot \mid g(e) > a\} \in \mathcal{F}_{X+\bot}$ for every $a \in \mathbb{R}$.

- If $a \geq 1$: $\{e \in X + \bot \mid g(e) > a\} = \emptyset \in \mathcal{F}_{X+\bot}$.
- If $a < 0$: $\{e \in X + \bot \mid g(e) > a\} = X + \bot \in \mathcal{F}_{X+\bot}$.
- If $0 \leq a < 1$: $\{e \in X + \bot \mid g(e) > a\}$.

The only nontrivial case is the last. It splits into two sets:

$$\begin{aligned}\{e \in X + \bot \mid g(e) > a\} &= \{e \in X + \bot \mid f_\bot(e)(A) > a\} \\ &= \{e \in X \mid f_\bot(e)(A) > a\} \cup \{\bot \mid f_\bot(e)(A) > a\} \\ &= \{e \in X \mid f(e)(A) > a\} \cup \{\bot \mid \delta_\bot(A) > a\}\end{aligned}$$

The first set is measurable in X (and hence $X + \bot$) since f is a Markov sub-Markov kernel by assumption. The second set is either $\emptyset \in \mathcal{F}_{X+\bot}$ or $\{\bot\} \in \mathcal{F}_{X+\bot}$. Since $\mathcal{F}_{X+\bot}$ is closed under countable unions, then $g^{-1}(B) \in \mathcal{F}_{X+\bot}$ for every $B \in \mathcal{B}([0,1])$. Thus, for all $A \in \mathcal{F}_{Y+\bot}$, the map $e \mapsto f_\bot(e)(A)$ is a measurable function.

Since both conditions hold, $f_\bot$ is a Markov kernel. sub-Markov kernel. If f is Markov, both cases above have total mass one, so $f_\bot$ is Markov. □

Lemma F.31 (Dirac return is a Markov kernel). *For any measurable space $(X, \mathcal{F}_X)$, the map*

$$\mathsf{return} : X \to \mathcal{D}(X + \bot), \qquad x \mapsto \delta_x.$$

is a Markov kernel (and hence a sub-Markov kernel).

Proof. We verify the two conditions for return to be a Markov kernel kernel conditions and unit total mass.

(1) Probability measure: We show for each $x \in X$, the map

$$A \mapsto \mathsf{return}(x)(A) = \delta_x(A)$$

is a probability measure.

Fix $x \in X$. δ_x is the Dirac measure at x, hence a probability measure on $X + \bot$. Therefore, for every $x \in X$, the map $A \mapsto \delta_x(A)$ is a probability measure, with $\delta_x(X + \bot) = 1$.

(2) Measurable function: We show that for each $A \in \mathcal{F}_{X+\bot}$, the map

$$x \mapsto \mathsf{return}(x)(A) = \delta_x(A)$$

is a measurable function.

Fix $A \in \mathcal{F}_{X+\bot}$. Define

$$g : X \to [0,1], \qquad g(x) := \delta_x(A).$$

We prove g is measurable by showing $g^{-1}(B) \in \mathcal{F}_X$ for every $B \in \mathcal{B}([0,1])$. It suffices to prove that for every $a \in \mathbb{R}$,

$$g^{-1}\big((a, \infty) \cap [0,1]\big) \in \mathcal{F}_X,$$

because the family $\{(a, \infty) \cap [0,1] : a \in \mathbb{R}\}$ generates $\mathcal{B}([0,1])$.

Because $g(e) \in [0,1]$ for all $e \in X + \bot$, we have for every $a \in \mathbb{R}$,

$$\begin{aligned}g^{-1}\big((a, \infty) \cap [0,1]\big) &= \{e \in X \mid g(e) \in (a, \infty) \cap [0,1]\} \\ &= \{e \in X \mid (g(e) \in (a, \infty)) \wedge (g(e) \in [0,1])\} \\ &= \{e \in X \mid (g(e) > a) \wedge (g(e) \in [0,1])\} \\ &= \{e \in X \mid g(e) > a\},\end{aligned}$$

where the last step uses that $g(e) \in [0,1]$ always holds. Hence it suffices to show that $\{e \in X \mid g(e) > a\} \in \mathcal{F}_X$ for every $a \in \mathbb{R}$.

- If $a \geq 1$: $\{x \in X \mid g(x) > a\} = \emptyset \in \mathcal{F}_X$.
- If $a < 0$: $\{x \in X \mid g(x) > a\} = X \in \mathcal{F}_X$.
- If $0 \leq a < 1$: $\{x \in X \mid g(x) > a\}$.

The only nontrivial case is the last. Using $g(x) = \delta_x(A)$ and the fact that $\delta_x(A) \in \{0, 1\}$ for all $x \in X$, we have for $0 \leq a < 1$:

$$\begin{aligned}\{x \in X \mid g(x) > a\} &= \{x \in X \mid \delta_x(A) > a\} \\ &= \{x \in X \mid \delta_x(A) = 1\} \\ &= \{x \in X \mid x \in A\} \\ &= A \cap X.\end{aligned}$$

Since $A \in \mathcal{F}_{X+\bot}$ and $\mathcal{F}_X = \{X \cap A \mid A \in \mathcal{F}_{X+\bot}\}$ is the subspace σ-algebra on $X \subseteq X + \bot$, it follows that $A \cap X \in \mathcal{F}_X$.

Therefore $\{x \in X \mid g(x) > a\} \in \mathcal{F}_X$ for all $a \in \mathbb{R}$, so $g^{-1}\big((a, \infty) \cap [0, 1]\big) \in \mathcal{F}_X$ for all a. Hence $g^{-1}(B) \in \mathcal{F}_X$ for every $B \in \mathcal{B}([0, 1])$, and g is measurable. □

Lemma F.32 (Zero kernel is a sub-Markov kernel). *Let $(X, \mathcal{F}_X)$ and $(Y, \mathcal{F}_Y)$ be measurable spaces. The constant-zero map*

$$f : X \to \mathcal{D}(Y), \qquad f(x)(A) := 0$$

is a Markov sub-Markov kernel.

Proof. We verify the two kernel conditions.

(1) Probability Subprobability measure. We show for each $x \in X$, the map

$$A \mapsto f(x)(A)$$

is a probability subprobability measure.

Fix any $x \in X$. Then $f(x)$ is the zero measure: it satisfies σ-additivity since

$$f(x)\left(\bigcup_i A_i\right) = 0 = \sum_i 0 = \sum_i f(x)(A_i)$$

for every pairwise disjoint family $(A_i)_i$, and total mass $f(x)(Y) = 0 \leq 1$.

(2) Measurable function. For any $A \in \mathcal{F}_Y$, the map $x \mapsto f(x)(A) = 0$ is the constant-zero function on X, hence measurable.

Thus both conditions hold, so f is a Markov sub-Markov kernel. □

Lemma F.33 (Composition of sub-Markov kernels). *Let (X, Σ_X), (Y, Σ_Y), and (Z, Σ_Z) be measurable spaces. If*

$$P : X \to \mathcal{D}(Y) \quad \text{and} \quad Q : Y \to \mathcal{D}(Z)$$

are Markov sub-Markov kernels, then their composition

$$(P \gg\!= Q)(x, A) \;:=\; \int_Y Q(y, A)\, P(x, dy)$$

defines a Markov sub-Markov kernel

$$P \gg\!= Q : X \to \mathcal{D}(Z).$$

Proof. SeeFor each $x \in X$, the map $A \mapsto (P \gg\!= Q)(x, A)$ is a measure by the standard integral construction. Its total mass is bounded by

$$(P \gg\!= Q)(x, Z) = \int_Y Q(y, Z)\, P(x, dy) \leq \int_Y 1\, P(x, dy) = P(x, Y) \leq 1.$$

For each $A \in \Sigma_Z$, measurability of $y \mapsto Q(y, A)$ and the standard measurability theorem for integration against a kernel imply that $x \mapsto \int_Y Q(y, A)\, P(x, dy)$ is measurable; see [30, Lemma 1.38]. Hence the composition is a sub-Markov kernel. □

Lemma F.34 (Typed strict Kleisli laws). *Let*

$$g : X \to \mathcal{D}(Y + \bot) \quad \textit{and} \quad h : Y \to \mathcal{D}(Z + \bot)$$

be sub-Markov kernels, and let $\mu \in \mathcal{D}(X + \bot)$. Then the following equalities of measures are well typed:

$$\delta_x \gg\!= g = g(x) \qquad (x \in X), \tag{1}$$

$$\mu \gg\!= \mathsf{return}_X = \mu, \tag{2}$$

$$(\mu \gg\!= g) \gg\!= h = \mu \gg\!= \big(\lambda x \in X.\, g(x) \gg\!= h\big). \tag{3}$$

Moreover, $\delta_\bot \gg\!= g = \delta_\bot$.

Proof. The left-unit and strictness equations follow by integration against a Dirac measure:

$$(\delta_x \gg\!= g)(A) = g_\bot(x)(A) = g(x)(A), \qquad (\delta_\bot \gg\!= g)(A) = g_\bot(\bot)(A) = \delta_\bot(A).$$

The $\bot$-extension of return_X is the identity Dirac kernel $u \mapsto \delta_u$ on $X + \bot$. Hence

$$(\mu \gg\!= \mathsf{return}_X)(A) = \int_{X+\bot} \delta_u(A)\, \mu(du) = \mu(A).$$

For associativity, define the ordinary-input kernel

$$k : X \to \mathcal{D}(Z + \bot), \qquad k(x) = g(x) \gg\!= h.$$

It is a sub-Markov kernel by Lemmas F.30 and F.33. For measurable $A \subseteq Z + \bot$, kernel integration gives

$$\begin{aligned}((\mu \gg\!= g) \gg\!= h)(A) &= \int_{X+\bot} \left(\int_{Y+\bot} h_\bot(y)(A)\, g_\bot(x, dy) \right) \mu(dx) \\ &= \int_{X+\bot} k_\bot(x)(A)\, \mu(dx) \\ &= (\mu \gg\!= k)(A).\end{aligned}$$

The middle equality holds for $x \in X$ by the definition of k. At $x = \bot$, its left side is $h_\bot(\bot)(A) = \delta_\bot(A)$, which is exactly $k_\bot(\bot)(A)$. □

F.4 Small-Step Semantics

Throughout this section, we denote by $e : \tau$ $e : \tau$ denotes a well-typed expression of type τ and implicitly assume that all sub-expressions are annotated with types, as well. Moreover, for the typed expression $e : \tau = \mathbf{uniform}(e_1 : \mathbf{float}[B_1; V_1], e_2 : \mathbf{float}[B_2; V_2]) : \mathbf{float}[B, V]$ with $B, B_1, B_2 \neq \top$ and $V_1, V_2 \neq \top$, we introduce an auxiliary statement $\mathtt{DC}_{B,B_1,B_2,V_1,V_2}$ and let We define the semantics of $\mathtt{DC}_{B,B_1,B_2,V_1,V_2}$ in such a way that it behaves like the discretized program in the last case of Figure 13 but resolves the whose subexpressions are fully type annotated. The dynamic-parameter transformation in Figure 13 denotes the displayed nest of core-language let-bindingsand the conditional choices in a single step, which will simplify our proof. We equip this statement with the following typing rule : , conditionals, Boolean conjunctions, finite equalities, and a discrete choice. No auxiliary one-step dispatch constructor is added to the language. The historical synchronous coupling below is disabled because its proof-only $\mathtt{DC}_{B,B_1,B_2,V_1,V_2}$ constructor does not describe the literal emitted target.

Equipped with these notions, we define the small-step semantics of our language:

Definition F.35. The small-step semantics $[\![e : \tau]\!] \in \mathcal{D}(Expr_\tau + \bot)$ is defined recursively on the structure of $e : \tau$ by the rules in Table 1 and Table 2.

The missing probability mass in $[\![e : \tau]\!]$ is the probability of encountering an error due to ill-formed parameters fed into distributions (e.g., $\mathbf{uniform}(0.8, 0.1)$). $[\![e : \tau]\!](\bot)$ is the probability of encountering an observation failure.

By Lemma F.36, ψ_s is measurable.
By construction, for all $\vec{r} \in A_{s;\tau_1}$:

$$e[s[\vec{r}]/x] = t[\psi_s(\vec{r})].$$

Define the measurable set of hole-assignments corresponding to A at skeleton t:

$$\begin{aligned} A_t &:= \{\vec{u} \in A_{t;\tau_2} \mid t[\vec{u}] \in A\} \\ &:= \{\vec{u} \in A_{t;\tau_2} \mid t[\vec{u}] \in A \cap Expr_{t;\tau_2}\} && (t[\vec{u}] \text{ ranges over } Expr_{t;\tau_2}) \end{aligned}$$

By Definition F.23, $A \cap Expr_{t;\tau_2} \in \mathcal{F}_{Expr_{t;\tau_2}}$, and by Lemma F.3, $A_t \in \mathcal{F}_{t;\tau_2}$. We compute:

$$\begin{aligned} R_s &= \{\vec{r} \in A_{s;\tau_1} \mid e[s[\vec{r}]/x] \in A\} \\ &= \{\vec{r} \in A_{s;\tau_1} \mid t[\psi_s(\vec{r})] \in A\} \\ &= \{\vec{r} \in A_{s;\tau_1} \mid \psi_s(\vec{r}) \in A_t\} \\ &= \psi_s^{-1}(A_t). \end{aligned}$$

Since ψ_s is measurable and $A_t \in \mathcal{F}_{t;\tau_2}$, we have $R_s = \psi_s^{-1}(A_t) \in \mathcal{F}_{s;\tau_1}$.
Since $R_s \in \mathcal{F}_{s;\tau_1}$ for every $s \in Skel_{\tau_1}$, we get $C \in \mathcal{F}_{Expr_{\tau_1}}$, and therefore

$$\mathrm{Sub}_{e,x}^{-1}(A) = C \cap Val_{\tau_1} \in \mathcal{F}_{Val_{\tau_1}}.$$

Hence $\mathrm{Sub}_{e,x}$ is measurable. □

Operational measurability obligations (open). The following kernel results are conditional on three facts for every rule in Tables 1 and 2: the syntactic case domains and side-condition regions form measurable partitions; every deterministic reconstruction or substitution map used under a Dirac measure is measurable; and every primitive sampling clause is a sub-Markov kernel on its parameter-validity region. The preceding lemmas establish representative substitution and reconstruction cases, but they do not discharge these obligations for every rule.

Lemma F.39 (Strict bind continuations, conditional). *For every occurrence of a monadic bind $\mu \gg\!= f$ given Assuming the operational measurability obligations above, every continuation in a bind $\mu \gg\!= f$ in the small-step operational semantics in Tables 1 and 2, $f : (Expr_{\tau_1} + \bot) \to \mathcal{D}(Expr_{\tau_2} + \bot)$ is a Markov kernel rules is a sub-Markov kernel*

$$f : Expr_{\tau_1} \to \mathcal{D}(Expr_{\tau_2} + \bot).$$

The operational bind uses its canonical extension $f_\bot : (Expr_{\tau_1} + \bot) \to \mathcal{D}(Expr_{\tau_2} + \bot)$.

Proof. We first prove that $f : Expr_{\tau_1} \to \mathcal{D}(Expr_{\tau_2} + \bot)$ is a Markov sub-Markov kernel. Let e be an expression whose evaluation rule contains a monadic bind $\mu \gg\!= f$. We proceed by case analysis on e:

- Consider $e = (\mathbf{let}\ y = x : \tau_1\ \mathbf{in}\ e_2 : \tau_2) : \tau_2$.
  This means proving that $\lambda x.\, \delta_{\mathbf{let}\ y=x\ \mathbf{in}\ e_2}$ is a Markov kernel (and hence a sub-Markov kernel). Formally, define

  $$f\colon Expr_{\tau_1} \to \mathcal{D}(Expr_{\tau_2} + \bot), \qquad f(x) = \delta_{\mathbf{let}\ y=x\ \mathbf{in}\ e_2}.$$

  We verify the two conditions for f to be a Markov kernel.

  (i) Probability measure: We show for each $x \in Expr_{\tau_1}$, the map $A \mapsto f(x)(A)$ is a probability measure.

  Fix $x \in Expr_{\tau_1}$. The map $A \mapsto f(x)(A)$ equals $A \mapsto \delta_{\mathbf{let}\ y=x\ \mathbf{in}\ e_2}(A)$, which satisfies:

  $$f(x)(A) = \begin{cases} 1 & \text{if } \mathbf{let}\ y = x\ \mathbf{in}\ e_2 \in A, \\ 0 & \text{otherwise.} \end{cases}$$

  A Dirac measure is a probability measure, so this condition holds.

Define the measurable set of hole-assignments corresponding to A at skeleton s':

$$\begin{aligned} A_{s'} &:= \{(\vec{r}, \vec{r}\,') \in A_{s';\tau_2} \mid s'[\vec{r}, \vec{r}\,'] \in A\} \\ &:= \{(\vec{r}, \vec{r}\,') \in A_{s';\tau_2} \mid s'[\vec{r}, \vec{r}\,'] \in A \cap Expr_{\tau_2}\} && \text{(Definition F.1)} \\ &:= \{(\vec{r}, \vec{r}\,') \in A_{s';\tau_2} \mid s'[\vec{r}, \vec{r}\,'] \in A \cap Expr_{s';\tau_2}\} && (s'[\vec{r}, \vec{r}\,'] \text{ ranges over } Expr_{s';\tau_2}) \end{aligned}$$

Since $A \cap Expr_{s';\tau_2} \in \mathcal{F}_{Expr_{s';\tau_2}}$, by Lemma F.21, $A_{s'} \in \mathcal{F}_{s';\tau_2}$.

Then:

$$\begin{aligned} R_s &= \{\vec{r} \in A_{s;\tau_1} \mid \textbf{let } y = s[\vec{r}] \textbf{ in } e_2 \in A\} \\ &= \{\vec{r} \in A_{s;\tau_1} \mid s'[(\vec{r}, \vec{c})] \in A\} \\ &= \{\vec{r} \in A_{s;\tau_1} \mid s'[\phi_{\vec{c}}(\vec{r})] \in A\} \\ &= \{\vec{r} \in A_{s;\tau_1} \mid \phi_{\vec{c}}(\vec{r}) \in A_{s'}\} \\ &= \phi_{\vec{c}}^{-1}(A_{s'}). \end{aligned}$$

Since $A_{s'} \in \mathcal{F}_{s';\tau_2}$ and $\phi_{\vec{c}}$ is measurable, we conclude

$$\phi_{\vec{c}}^{-1}(A_{s'}) \in \mathcal{F}_{s;\tau_1},$$

and therefore $B \cap Expr_{s;\tau_1} \in \mathcal{F}_{Expr_{s;\tau_1}}$ for all $s \in Skel_{\tau_1}$, giving $B \in \mathcal{F}_{Expr_{\tau_1}}$ as required.

Hence, $\{x \in Expr_{\tau_1} \mid g(x) > a\} \in \mathcal{F}_{Expr_{\tau_1}}$ for all $a \in \mathbb{R}$, so $g^{-1}((a, \infty) \cap [0, 1]) \in \mathcal{F}_{Expr_{\tau_1}}$ for all $a \in \mathbb{R}$. Hence $g^{-1}(B) \in \mathcal{F}_{Expr_{\tau_1}}$ for every $B \in \mathcal{B}([0, 1])$, and g is measurable.

Thus, $f : Expr_{\tau_1} \to \mathcal{D}(Expr_{\tau_2} + \bot)$ is a Markov kernel, and hence a sub-Markov kernel.

- All other cases analogous. For every remaining rule, measurability of its case domain, reconstruction maps, and primitive kernels is one of the stated operational measurability obligations. Those obligations imply the required ordinary-input continuation is a sub-Markov kernel.

By Lemma F.30, the canonical $\bot$-extension $f_\bot : (Expr_{\tau_1} + \bot) \to \mathcal{D}(Expr_{\tau_2} + \bot)$ is also a Markov kernel , as required$f_\bot$ is a sub-Markov kernel and is the kernel integrated by the strict bind. □

Lemma F.40 (Small-step kernel, conditional). *For Assuming the operational measurability obligations above, for each type τ , the small-step operational semantics $[\![\cdot]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ defines a Markov semantics*

$$[\![\cdot]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$$

is a sub-Markov kernel.

By induction on the typing derivation of $e : \tau$. We show in each case that the map $e \mapsto [\![e : \tau]\!]$ is a Markov kernel.

Consider $e = (\textbf{let } y = x : \tau_1 \textbf{ in } e_2 : \tau_2) : \tau_2$. The small-step semantics is:

If $e_1 = v$, By Lemma F.31 and Lemma F.38, $e \mapsto \delta_{e_2[v/y]:\tau_2}$ is a Markov kernel . Hence, $[\![\textbf{let } x = e_1 \textbf{ in } e_2 : \tau_2]\!] : Expr_{\tau_2} \to \mathcal{D}(Expr_{\tau_2} + \bot)$ is a Markov kernel.

By

Proof. Proceed by structural induction through the operational rules. On each measurable case domain, deterministic clauses are Dirac kernels induced by the assumed measurable reconstruction map, primitive clauses are the assumed sub-Markov kernels, and divergence is the zero kernel from Lemma F.32. A bind clause first uses the induction hypothesis , is a Markov kernel. By Lemma F.39, is a Markov kernel. Hence

by Lemma F.33, their composition is a Markov kernel and then composes it with the canonical extension $f_\perp$ of the ordinary-input continuation supplied by Lemma F.39.

Thus, $[\![\textbf{let } x = e_1 \textbf{ in } e_2 : \tau_2]\!] : Expr_{\tau_2} \to \mathcal{D}(Expr_{\tau_2} + \perp)$ is a Markov kernel.

Consider $e = \textbf{diverge} : \tau$.

The small-step semantics is:

By Lemma F.32, is a Markov kernel.

Thus, $[\![\textbf{diverge} : \tau]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \perp)$ is a Markov kernel.

All other cases are analogous. This composition is a sub-Markov kernel by Lemmas F.30 and F.33. Combining the branch kernels over the assumed measurable partition proves the claim. □

F.5 Small-Step Coupling Semantics

Weak compiler-simulation obligation (open). Let $\longrightarrow_{\mathrm{a}}$ contain only deterministic administrative target reductions: let substitution, finite equality, Boolean conjunction, and conditional selection. Write $\Longrightarrow_{\mathrm{a}}$ for its reflexive transitive closure. A sound proof for the literal generated term must construct a measurable relation R_τ containing $(e, \mathcal{D}[\![e]\!])$ and closed under a source probabilistic step matched by finitely many target administrative steps, one target probabilistic step, and finitely many further administrative steps. The symmetric clause must permit target administrative steps while the source stutters. Related terminal Boolean values must be equal, and both sides must assign the same mass to observation failure. Establishing this weak simulation remains open.

Even such a weak relation is not by itself a terminal theorem. The conditional result below separately assumes that generated administrative reduction is normalizing and that iteration of the measurable target weak-step kernel has the same terminal limit as iteration of the ordinary target small-step kernel.

F.6 Big-Step Semanticsand Correctness of Discretization

We extend both of the Define the ordinary-input kernel

$$S_\tau(e') = [\![e' : \tau]\!].$$

We extend the ordinary small-step semantics canonically to an n-step semantics by explicitly integrating the canonical bottom-preserving extension:

$$\langle e : \tau\rangle_n = \begin{cases} \delta_{e:\tau} & \text{if } n = 0 \\ \lambda A. \displaystyle\int_{Expr_\tau + \perp} (S_\tau)_\perp(e')(A)\, \langle e : \tau\rangle_{n-1}(de') & \text{if } n > 0\,. \end{cases}$$

As a sanity check, we show that the n-step semantics defines a Markov sub-Markov kernel.

Lemma F.41 (n-step kernel, conditional). *For all $n \in \mathbb{N}$, $\langle\cdot\rangle_n : Expr_\tau \to \mathcal{D}(Expr_\tau + \perp)$ is a Markov Under the operational measurability obligations, for all $n \in \mathbb{N}$, $\langle\cdot\rangle_n : Expr_\tau \to \mathcal{D}(Expr_\tau + \perp)$ is a sub-Markov kernel.*

Proof. By induction on n.

Base case: $n = 0$. By definition,

$$\langle e : \tau\rangle_n = \delta_{e:\tau}$$

i.e. the map $e \mapsto \delta_e$. By Lemma F.31, this is a Markov kernel, and hence a sub-Markov kernel.

Inductive step: $n > 0$. By definition,

$$\langle e : \tau\rangle_n = \langle e : \tau\rangle_{n-1} \gg\!= \lambda e'.[\![e' : \tau]\!]$$

By the I.H., $\langle\cdot\rangle_{n-1} : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a Markov sub-Markov kernel. By Lemma F.40, $\lambda e'.[\![e' : \tau]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a Markov sub-Markov kernel. By Lemma F.30, its $\bot$-extension from $(Expr_\tau + \bot)$ to $\mathcal{D}(Expr_\tau + \bot)$ is also a Markov sub-Markov kernel. By Lemma F.33, $\langle e : \tau\rangle_{n-1} \gg\!= \lambda e'.[\![e' : \tau]\!]$ is a Markov sub-Markov kernel. □

For all $n \in \mathbb{N}$, $\langle\cdot,\cdot\rangle_n : P_\tau \to \mathcal{D}(P_\tau + \bot)$ is a Markov kernel.

Analogous to proof of Lemma F.41.

For every type τ, we denote by $Vals_\tau$ the set of values of type τ and equip it with a suitable measurable sub-space of the measurable space over $Expr_\tau + \bot$ let $T_\tau = Vals_\tau + \bot$ be the measurable subspace of terminal outcomes (cf. Lemma F.27.1). Moreover, let $V_\tau = \{(v : \tau, \mathcal{D}[\![v]\!] : \mathcal{D}[\![\tau]\!]) \mid v : \tau\}$. For $e : \tau$, define the restriction of its n-step semantics to terminal outcomes by

$$\nu_n^e(A) = \langle e : \tau\rangle_n(A) \qquad (A \in \mathcal{F}_{T_\tau}).$$

Each ν_n^e is a subprobability measure. The full n-step measures on expressions need not be increasing: a deterministic step removes mass from the singleton containing its source expression.

Lemma F.42 (Increasing terminal restrictions). *For every $e : \tau$, $n \in \mathbb{N}$, and measurable terminal event $A \in \mathcal{F}_{T_\tau}$,*

$$\nu_n^e(A) \le \nu_{n+1}^e(A).$$

Proof. Let $K_\tau = (S_\tau)_\bot$ be the bottom-extended one-step kernel. Terminal outcomes are absorbing: $K_\tau(x) = \delta_x$ for $x \in T_\tau$. Thus, for every $x \in Expr_\tau + \bot$ and measurable $A \subseteq T_\tau$, $K_\tau(x)(A) \ge \mathbf{1}_A(x)$. Integrating this inequality against $\langle e : \tau\rangle_n$ gives

$$\nu_{n+1}^e(A) = \int K_\tau(x)(A)\, \langle e : \tau\rangle_n(dx) \ge \nu_n^e(A).$$

□

Definition F.43 (Big-step semantics). For $e : \tau$, define a set function on terminal outcomes by

$$\langle e : \tau\rangle(A) = \lim_{n\to\infty} \nu_n^e(A), \qquad A \in \mathcal{F}_{T_\tau}.$$

Lemma F.44 (Big-step semantics is well-defined). *Conditional on the operational measurability obligations, the set function in Definition F.43 is a subprobability measure and, as a function of its initial expression, a sub-Markov kernel into T_τ.*

Proof. By Lemma F.42, each limit exists and equals a supremum. If $(A_j)_{j\in\mathbb{N}}$ are pairwise disjoint measurable subsets of T_τ, then monotone convergence on $\mathbb{N}$ with counting measure gives

$$\begin{aligned}\langle e : \tau\rangle\Big(\bigcup_j A_j\Big) &= \lim_n \sum_j \nu_n^e(A_j) \\ &= \sum_j \lim_n \nu_n^e(A_j) = \sum_j \langle e : \tau\rangle(A_j).\end{aligned}$$

Also $\langle e : \tau\rangle(T_\tau) \le 1$. For a fixed measurable terminal event, the finite-step kernel lemma makes its probability measurable in the initial expression, and the pointwise limit of these measurable functions is measurable. □

No coupled finite-step or big-step transfer is claimed here. Such a transfer additionally requires the open weak compiler simulation and a proof that restriction to terminal outcomes commutes with component pushforward.

Proposition F.45 (Conditional closed Boolean semantic preservation). *Let $e : \mathbf{bool}$. Then $\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle$. $e : \mathbf{bool} \in Expr^{\cup}_{\mathbf{bool}}$ be closed. Assume a measurable weak target kernel and a measurable weak coupling whose marginals are the source step and target weak step, respectively. Assume also that generated administrative reduction is normalizing, that the terminal restrictions of the coupling iterates have an increasing*

```
let x1 = uniform(0,1) in
let x2 = if x1 < 0.56 then uniform(0,2) else uniform(0,3) in
let x3 = if x2 < 0.92 then uniform(0,4) else uniform(0,5) in
let x4 = if x3 < 0.47 then uniform(0,6) else uniform(0,7) in
let x5 = if x4 < 0.51 then uniform(0,8) else uniform(0,9) in
x5 < 0.5
```

Fig. 28. Conditional Independent (Figure 36a): Program of size 5. Each variable depends on its immediate predecessor variable via a conditional branch. Each uniform distribution is distinct.

```
let x1 = uniform(0,1) in
let x2 = if x1 < 0.56 then uniform(0,2) else uniform(0,3) in
let x3 = if x1 < 0.45 then uniform(0,4) else uniform(0,5) in
let x4 = if x1 < 0.18 then uniform(0,6) else uniform(0,7) in
let x5 = if x1 < 0.48 then uniform(0,8) else uniform(0,9) in
x5 < 0.5
```

Fig. 29. Conditional Independent Fork (Figure 36b): Program of size 5. Each variable x_i depends on the same predecessor variable x_1 through a conditional branch. This creates a fork-shaped dependency pattern in which x_1 serves as the sole parent of all subsequent variables. Each uniform distribution is distinct.

limit, and that the target projection of this limit is the ordinary target terminal semantics. If related terminal Boolean values agree and the coupling preserves observation-failure mass, then $\langle e : \mathbf{bool} \rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool} \rangle$.

Proof. By the terminal-limit compatibility premises, the limiting coupling has the two displayed ordinary terminal semantics as its component pushforwards. Agreement of related terminal Booleans and preservation of observation-failure mass make those pushforwards equal on true, false, and $\bot$.

□

G Benchmarks

G.1 Asymptotic Scaling

G.1.1 Code Sketches for Scaling Benchmarks. This section contains code snippets of the programs used in the scaling experiments in Section 7.2. We implemented a Python script that automatically generates these programs based on configurable parameters such as program size, guard structure, and dependency pattern. For reproducibility, we used a fixed random seed to generate each program. All r_i values in the generated programs, where $0 \leq r_i \leq 1$ is randomly chosen, are rounded to two decimal places.

All of these programs follow the same structural pattern: they define a sequence of random variables $X_1, \ldots, X_n$, where each X_i is drawn from a uniform distribution whose parameters depend on the value of a single predecessor variable through a conditional branch of the form

$$X_i = \begin{cases} \mathrm{Unif}(a_i, b_i), & X_{\pi(i)} < r_i, \\ \mathrm{Unif}(a_i', b_i'), & \text{otherwise}, \end{cases}$$

for some parent-selection function $\pi : \{2, \ldots, n\} \to \{1, \ldots, i-1\}$. Thus each program is a Bayesian network of conditionally independent nodes, differing only in the choice of $\pi(i)$ that determines the dependency pattern.

The programs in Figure 28 follow the scheme depicted above where the program size is n, i.e., the number of comparisons performed is n. In Figure 29, each x_i does not necessarily depend on x_{i-1} but on some randomly chosen predecessor[3]. In Figures 30 and 31, we additionally choose at random whether the guards of the conditionals and the branches are either some predecessor variable x_i or a **uniform**-sampling instruction.

[3]We generate these programs in a randomized manner with a fixed seed for reproducibility